

Analisi Numerica

Sebastiano Boscarino

April 14, 2026

Metodi iterativi e diretti

Supponiamo di avere $A \in \mathbb{R}^{n \times n}$ (matrice non singolare), $b \in \mathbb{R}^n$ (vettore colonna) e di dover risolvere il problema $Ax = b$ avente soluzione unica x^* . A tal proposito si può utilizzare la fattorizzazione LU con pivoting. Il costo computazionale è in generale di $O(n^3/3)$ operazioni moltiplicative. Questo diventa proibitivo se n è particolarmente elevato.

L'idea dei metodi iterativi è quella di ottenere una successione di vettori $x^{(k)} \rightarrow x^*$, in modo che per $k \ll n$ si abbia $x^{(\bar{k})} \approx x^*$. In generale, la soluzione non è ottenuta esattamente come nei metodi diretti in un numero finito di operazioni (in aritmetica esatta), ma come limite, accontentandosi di poche iterazioni ognuna con costo quadratico. Quindi il costo totale sarà dell'ordine $O(\bar{k} \cdot n^2)$.

Metodi iterativi stazionari

Supponiamo che la matrice non singolare $A \in \mathbb{C}^{n \times n}$ sia tale che $A = M - N$, con M non singolare.

Allora, se $Ax = b$, necessariamente $Mx - Nx = Ax = b$. Di conseguenza, da $Mx = Nx + b$, moltiplicando ambo i membri per M^{-1} , si ha $x = M^{-1}Nx + M^{-1}b$.

Viene quindi naturale utilizzare $x^{(k+1)} = M^{-1}Nx^{(k)} + M^{-1}b$. La matrice $P = M^{-1}N$ si dice di iterazione e non dipende, come pure b , dall'indice di iterazione k . Per questo motivo tali metodi si chiamano iterativi stazionari.

Teorema

Condizione necessaria e sufficiente per la convergenza del metodo iterativo con matrice di iterazione P è

$$\rho(P) < 1,$$

essendo $\rho(P)$ il raggio spettrale della matrice P , ossia

$$\rho(P) = \max_{1 \leq i \leq n} |\lambda_i|,$$

con λ_i autovalori di P .

Esempio $A = D - E - F$

Risulta utile anche scrivere la matrice A come $A = D - E - F$ dove

- D è una matrice diagonale,
- E è triangolare inferiore con elementi diagonali nulli,
- F è triangolare superiore con elementi diagonali nulli.

È immediato osservare che tale scrittura esiste ed è unica. Tramite questo splitting di matrici descriveremo i metodi iterativi stazionari più noti e sarà particolarmente semplice la loro implementazione su computer.

Metodo di Jacobi e di Gauss-Seidel

Nel caso del metodo di Jacobi si ha

$$M = D, \quad N = E + F$$

e quindi

$$P = M^{-1}N = D^{-1}(E + F) = D^{-1}(D - D + E + F) =$$

$$D^{-1}(D - A) = I - D^{-1}A$$

Si osservi che se D è non singolare allora il metodo di Jacobi, almeno in questa versione di base, non può essere utilizzato visto che in (2) non ha senso la scrittura D^{-1} .

Il metodo di Gauss-Seidel è definito quale metodo stazionario in cui

$$M = D - E, \quad N = F$$

e quindi

$$P = M^{-1}N = (D - E)^{-1}F$$

Esempio A, E, F

```
>> A = [1, 2, 3, 4; 5, 6, 7, 2; 8, 9, 1, 2; 3, 4, 5, 1]
>> E = -(tril(A) - diag(diag(A)))
>> F = -(triu(A) - diag(diag(A)))
```

Metodo di Jacobi

Da $Ax = b$, con $A \in \mathbb{R}^{n \times n}$ abbiamo

$$\sum_{j < i} a_{i,j} x_j + a_{i,i} x_i + \sum_{j > i} a_{i,j} x_j = \sum_j a_{i,j} x_j = b_i, \quad i = 1, \dots, n$$

e quindi evidenziando x_i , supposto $a_{i,i} \neq 0$ deduciamo

$$x_i = \frac{1}{a_{i,i}} \left(b_i - \left(\sum_{j < i} a_{i,j} x_j + \sum_{j > i} a_{i,j} x_j \right) \right), \quad i = 1, \dots, n$$

Quindi, note delle approssimazioni della soluzione $x^{(k)} = \{x_j^{(k)}\}$ per $j = 1, \dots, n$, è naturale introdurre il metodo iterativo

$$x_i^{(k+1)} = \frac{1}{a_{i,i}} \left(b_i - \left(\sum_{j < i} a_{i,j} x_j^{(k)} + \sum_{j > i} a_{i,j} x_j^{(k)} \right) \right), \quad i = 1, \dots, n,$$

che si vede facilmente essere il metodo di Jacobi.

Metodo di Gauss-Seidel

Vediamo perché questa derivazione. Da $Ax = b$, con $A \in \mathbb{R}^{n \times n}$ abbiamo

$$\sum_{j < i} a_{i,j} x_j + a_{i,i} x_i + \sum_{j > i} a_{i,j} x_j = \sum_j a_{i,j} x_j = b_i, \quad i = 1, \dots, n$$

e quindi evidenziando x_i , supposto $a_{i,i} \neq 0$ deduciamo

$$x_i = \frac{1}{a_{i,i}} \left(b_i - \left(\sum_{j < i} a_{i,j} x_j + \sum_{j > i} a_{i,j} x_j \right) \right), \quad i = 1, \dots, n$$

Quindi, note delle approssimazioni della soluzione $x^{(k)} = \{x_j^{(k)}\}$ per $j > i$, e già calcolate $x^{(k+1)} = \{x_j^{(k+1)}\}$ per $j < i$, è naturale introdurre il metodo iterativo

$$x_i^{(k+1)} = \frac{1}{a_{i,i}} \left(b_i - \left(\sum_{j < i} a_{i,j} x_j^{(k+1)} + \sum_{j > i} a_{i,j} x_j^{(k)} \right) \right), \quad i = 1, \dots, n.$$

SOR

Definizione (Successive over relaxation, SOR)

Sia $A = D - E - F \in \mathbb{R}^{n \times n}$ dove:

- D è una matrice diagonale, non singolare,
- E è triangolare inferiore con elementi diagonali nulli,
- F è triangolare superiore con elementi diagonali nulli.

Sia fissato $x^{(0)} \in \mathbb{R}^n$ e $\omega \in \mathbb{R} \setminus \{0\}$. Il metodo SOR genera $\{x^{(k)}\}$ con

$$x^{(k+1)} = M^{-1}Nx^{(k)} + M^{-1}b$$

dove

$$M = \frac{1}{\omega}(D - \omega E), \quad N = \frac{1}{\omega}((1 - \omega)D + \omega F).$$

e

$$(D - \omega E)x^{(k+1)} = \left([(1 - \omega)D + \omega F]x^{(k+1)} \right) x^{(k)} + \omega b,$$

con

$$(I - \omega D^{-1}E)x^{(k+1)} = \left([(1 - \omega)I + \omega D^{-1}F]x^{(k+1)} \right) x^{(k)} + \omega D^{-1}b.$$

SOR

Nota (cf. [1, p.555])

Il metodo SOR deriva dal descrivere esplicitamente una iterazione di Gauss-Seidel

$$z_i^{(k+1)} = \frac{1}{a_{ii}} \left[b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right]$$

e sostituirla con la combinazione convessa

$$x_i^{(k+1)} = \omega z_i^{(k+1)} + (1 - \omega) x_i^{(k)}$$

cioè

$$x_i^{(k+1)} = \omega \frac{1}{a_{ii}} \left[b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right] + (1 - \omega) x_i^{(k)}$$

Alcune definizioni

Ricordiamo che A è a predominanza diagonale (per righe) se per ogni $i = 1, \dots, n$ risulta

$$|a_{i,i}| \geq \sum_{j=1, j \neq i}^n |a_{i,j}|$$

e per almeno un indice s si abbia

$$|a_{s,s}| > \sum_{j=1, j \neq s}^n |a_{s,j}|$$

Se per tutti gli indici s si ha $|a_{s,s}| > \sum_{j=1, j \neq s}^n |a_{s,j}|$ allora la predominanza diagonale è in senso stretto. Ad esempio la matrice

$$A = \begin{pmatrix} 4 & -4 & 0 \\ -1 & 4 & -1 \\ 0 & -4 & 4 \end{pmatrix}$$

è a predominanza diagonale per righe (non stretta).

Teoremi di convergenza

- Sia A una matrice quadrata a predominanza diagonale per righe, in senso stretto. Allora il metodo di Jacobi converge alla soluzione di $Ax = b$, qualsiasi sia il punto $x^{(0)}$ iniziale.
- Sia A a predominanza diagonale per righe, in senso stretto. Allora il metodo di Gauss-Seidel risulta convergente, qualsiasi sia il punto $x^{(0)}$ iniziale.

Alcune definizioni

La matrice A è simmetrica se $A = A^T$. Una matrice A è definita positiva se ha tutti gli autovalori positivi. Ad esempio, la matrice

$$A = \begin{pmatrix} 4 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 4 \end{pmatrix}$$

è simmetrica e definita positiva.

Alcuni teoremi di convergenza

Ricordiamo che equivalentemente una matrice A è definita positiva se $x^T A x > 0$, per ogni $x \neq 0$.

Teorema Sia A una matrice simmetrica, non singolare con elementi principali $a_{i,i} \neq 0$. Allora il metodo di Gauss-Seidel è convergente per qualsiasi scelta del punto iniziale $x^{(0)}$ se e solo se A è definita positiva.

Alcuni teoremi di convergenza

Teorema Sia $A \in \mathbb{R}^{n \times n}$ una matrice quadrata. Condizione necessaria affinché il metodo SOR converga globalmente alla soluzione di $Ax = b$ è che $\omega \in (0, 2)$.

Teorema Sia $A \in \mathbb{R}^{n \times n}$ definita positiva, $\omega \in (0, 2)$. Allora il metodo SOR converge.

Test di arresto: criterio dello step

Fissata dall'utente una tolleranza tol , si desidera interrompere il processo iterativo quando $\|x^* - x^{(k)}\| \leq \text{tol}$.

Definition (Criterio dello step)

Il criterio dello step consiste nell'interrompere il metodo iterativo che genera la successione $\{x^{(k)}\}$ alla $(k + 1)$ -esima iterazione qualora

$$\|x^{(k+1)} - x^{(k)}\| \leq \text{tol}$$

(dove tol è una tolleranza fissata dall'utente).

Test di arresto: criterio del residuo

Definition (Criterio del residuo)

Il criterio del residuo consiste nell'interrompere il metodo iterativo che genera la successione $\{x^{(k)}\}$ alla $(k+1)$ -esima iterazione qualora

$$\|r^{(k)}\| \leq \text{tol}$$

dove tol è una tolleranza fissata dall'utente, e

$$r^{(k)} = b - Ax^{(k)}.$$

In generale conviene tuttavia effettuare un test d'arresto sul residuo normalizzato:

$$\|r^{(k)}\| / \|r^{(0)}\| \leq \text{tol},$$

oppure quando

$$\|r^{(k)}\| / \|b\| \leq \text{tol}, \quad x^{(0)} = 0.$$

In quest'ultimo caso si ha il seguente controllo sull'errore relativo commesso:

Teorema

Sia $Ax^* = b$ con A non singolare e $\|\cdot\|$ una norma naturale. Sia $\{x^{(k)}\}$ la successione generata da un metodo iterativo. Allora

$$\frac{\|x^{(k)} - x^*\|}{\|x^*\|} = \frac{\|e^{(k)}\|}{\|x^*\|} \leq \kappa(A) \frac{\|r^{(k)}\|}{\|b\|} \leq \kappa(A) \text{tol},$$

dove $\kappa(A)$ è il numero di condizionamento di A .

Nota: Dal teorema si evince che il criterio $\frac{\|r^{(k)}\|}{\|b\|} \leq \text{tol}$ non offre indicazioni su $\frac{\|x^{(k)} - x^*\|}{\|x^*\|}$ per $\kappa(A) \gg 1$.

Test di arresto: criterio del residuo - Dimostrazione

Dimostrazione. Se $b = Ax^*$ allora, essendo A invertibile, abbiamo

$$e^{(k)} = x^* - x^{(k)} = A^{-1}A(x^* - x^{(k)}) = A^{-1}(b - Ax^{(k)}) = A^{-1}r^{(k)}$$

e quindi, essendo $\|\cdot\|$ una norma naturale,

$$\|e^{(k)}\| \leq \|A^{-1}\| \|r^{(k)}\|;$$

Inoltre, poiché $b = Ax^*$ abbiamo $\|b\| \leq \|A\| \|x^*\|$ e quindi

$$\frac{1}{\|x^*\|} \leq \frac{\|A\|}{\|b\|}.$$

Di conseguenza, posto $\kappa(A) = \|A\| \|A^{-1}\|$, se $x^* \neq 0$ ricaviamo

$$\frac{\|e^{(k)}\|}{\|x^*\|} \leq \frac{\|A\|}{\|b\|} \cdot \|A^{-1}\| \|r^{(k)}\| = \kappa(A) \frac{\|r^{(k)}\|}{\|b\|}.$$

Metodi iterativi alternativi

Sia A una matrice quadrata in $\mathbb{R}^{n \times n}$, $b \in \mathbb{R}^n$, $x \in \mathbb{R}^n$.

Consideriamo un metodo iterativo basato sullo *splitting* di A :

$$A = P - (P - A),$$

con P matrice non singolare e detta *precondizionamento* di A . Allora abbiamo

$$Px = (P - A)x + b,$$

con $M = P^{-1}(P - A) = I - P^{-1}A$ matrice di iterazione associata al metodo

$$P(x^{(k+1)} - x^{(k)}) = r^{(k)}, \quad r^{(k)} = b - Ax^{(k)},$$

$r^{(k)}$ il vettore residuo all'iterata k . Una generalizzazione di questo metodo iterativo è:

$$P(x^{(k+1)} - x^{(k)}) = \alpha_k r^{(k)},$$

con parametro $\alpha_k \neq 0$ di rilassamento (o di accelerazione) che può cambiare ad ogni iterazione e serve a migliorare le proprietà di convergenza della successione $\{x^{(k)}\}$.

Si ottengono i metodi di Richardson stazionari con α fissato ad ogni iterazione k .

Riscriviamo i metodi di Richardson non-stazionari in una forma di interesse computazionale.

Posto:

$$Pz^{(k)} = r^{(k)},$$

detto $z^{(k)}$ residuo preconditionato, quindi la nuova iterazione è definita come

$$x^{(k+1)} = x^{(k)} + \alpha_k z^{(k)}, \quad \alpha_k \neq 0, \quad k \geq 0$$

e

$$r^{(k+1)} = b - Ax^{(k+1)} = r^{(k)} - \alpha_k Az^{(k)}$$

quindi la nuova iterazione è definita come $x^{(k+1)} = x^{(k)} + \alpha_k z^{(k)}$.

Riassumendo:

- ① risolvere il sistema lineare: $Pz^{(k)} = r^{(k)}$;
- ② calcolare il parametro di accelerazione α_k ;
- ③ aggiornare la soluzione: $x^{(k+1)} = x^{(k)} + \alpha_k z^{(k)}$;
- ④ aggiornare il residuo: $r^{(k+1)} = r^{(k)} - \alpha_k A z^{(k)}$;

RIASSUMENDO: I metodi di Richardson stazionari

Sia P una matrice di preconditionamento:

Definition (Metodo di Richardson stazionario)

Fissato α , un metodo di Richardson stazionario, con matrice di preconditionamento P , verifica

$$P(x^{(k+1)} - x^{(k)}) = \alpha r^{(k)}.$$

dove il residuo alla k -esima iterazione è

$$r^{(k)} = b - Ax^{(k)}.$$

I metodi di Richardson non stazionari

Definition (Metodo di Richardson non stazionario)

Sia α_k coefficienti dipendenti dalla iterazione k , un metodo di Richardson non stazionario, con matrice di preconditionamento P , verifica

$$P(x^{(k+1)} - x^{(k)}) = \alpha_k r^{(k)}.$$

Nota: Tipicamente la matrice P viene scelta cosicché la soluzione di

$$P(x^{(k+1)} - x^{(k)}) = \alpha_k r^{(k)}$$

non sia computazionalmente costosa. Ad esempio, P può essere diagonale o triangolare.

I metodi di Richardson

I metodi di Jacobi e di Gauss-Seidel, SOR, sono metodi di Richardson stazionari.

In generale, i metodi del tipo $Mx^{(k+1)} = Nx^{(k)} + b$, con M invertibile e $A = M - N$ sono di Richardson stazionari con $P = M$, $\alpha = 1$.

Dimostrazione. Sia, come da ipotesi,

$$Mx^{(k+1)} = Nx^{(k)} + b.$$

Essendo $r^{(k)} = b - Ax^{(k)}$,

$$M(x^{(k+1)} - x^{(k)}) = Nx^{(k)} + b - Mx^{(k)} = b - Ax^{(k)} = r^{(k)}.$$

J $M = D$ per GS $M = D - E$.

I metodi di Richardson

Nota: Per l'asserto precedente, poiché i metodi di Jacobi e di Gauss-Seidel, SOR, verificano

$$M(x^{(k+1)} - x^{(k)}) = r^{(k)},$$

necessariamente sono metodi di Richardson stazionari, con $\alpha = 1$ e matrice di preconditionamento $P = M$.

I metodi di tipo Richardson $P(x^{(k+1)} - x^{(k)}) = \alpha(b - Ax^{(k)})$ sono spesso usati cercando di selezionare i parametri α così da rendere più veloce la convergenza del metodo.

Nel caso $A = A^T$, tale parametro ottimale è

$$\alpha = \frac{2}{\lambda_{\min} + \lambda_{\max}}.$$

Per quanto riguarda i metodi di Richardson preconditionati e non stazionari, un classico esempio è il metodo del gradiente classico che vedremo in seguito.

I metodi di discesa

Sia A una matrice simmetrica definita positiva. Si osserva che se x^* è l'unica soluzione di $Ax = b$ allora è pure il minimo del funzionale dell'energia

$$\phi(x) = \frac{1}{2}x^T Ax - x^T b, \quad x \in \mathbb{R}^n$$

detta **energia del sistema**. In effetti, il gradiente di $\phi(x)$:

$$\nabla(\phi(x)) = \text{grad}(\phi(x)) = Ax - b = 0 \iff Ax^* = b.$$

Quindi invece di calcolare la soluzione del sistema lineare, intendiamo calcolare il minimo del funzionale ϕ , partendo da un punto $x_0 \in \mathbb{R}$.

I metodi di discesa

La direzione ottimale, congiungente x_0 con x^* , non è ovviamente nota a priori. Quindi l'idea è di muoversi verso una direzione $p^{(0)}$, partendo da x_0 , e su questa fissare un punto $x^{(1)}$, da quale ripetere il procedimento fino a convergenza.

Un generico metodo di discesa consiste nel generare una successione

$$x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}$$

dove $p^{(k)}$ è una direzione fissata secondo qualche criterio e α_k il valore della lunghezza del passo.

Lo scopo ovviamente è che

$$\phi(x^{(k+1)}) < \phi(x^{(k)}),$$

e che il punto x^* , in cui si ha il minimo di ϕ , venga calcolato rapidamente.

I metodi di discesa

Metodo del gradiente classico

Il metodo del gradiente classico è un metodo di discesa

$$x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}$$

con α_k e $p^{(k)}$ scelti così da ottenere la massima riduzione del funzionale dell'energia ϕ a partire dal punto $x^{(k)}$.

Differenziando $\phi(x) = \frac{1}{2}x^T Ax - x^T b$, $x \in \mathbb{R}^n$:

$$\nabla \phi(x^{(k)}) = Ax^{(k)} - b = -r^{(k)},$$

quindi la direzione del gradiente coincide con quella del residuo $r^{(k)}$.

Al passo k quindi anche il metodo del gradiente, come quello di Richardson, si muovono lungo la direzione

$$p^{(k)} = r^{(k)} = -\nabla \phi(x^{(k)}).$$

$$x^{(k+1)} = x^{(k)} - \alpha_k \nabla \phi(x^{(k)}).$$

Per il calcolo di α_k abbiamo:

$$\begin{aligned}\phi(x^{k+1}) &= \frac{1}{2}(x^{(k+1)})^T A x^{(k+1)} - (x^{(k+1)})^T b \\ &= \frac{1}{2}(x^{(k)} + \alpha_k r^{(k)})^T A (x^{(k)} + \alpha_k r^{(k)}) - (x^{(k)} + \alpha_k r^{(k)})^T b.\end{aligned}$$

Essendo una funzione quadrati derivando rispetto ad α_k , e imponendo l'annullamento della derivata, abbiamo

$$\alpha_k = \frac{r^{(k)T} r^{(k)}}{(r^{(k)})^T A r^{(k)}}$$

dove α_k è solo una funzione del residuo al passo k .

Nota: Con qualche facile conto si vede che è un metodo di Richardson non stazionario con $P = I$ e parametro α_k definito come sopra e che dipende solo dal residuo.

Nota: La convergenza del gradiente è lenta qualora si abbiano alti numeri di condizionamento

Il metodo del gradiente da luogo al seguente algoritmo. Dato $x^{(0)} \in \mathbb{R}$, per $k = 0, 1, \dots$ fino a convergenza, si calcola

$$p^{(k)} = r^{(k)} = b - Ax^{(k)},$$

$$\alpha_k = \frac{r^{(k)T} r^{(k)}}{(r^{(k)})^T A r^{(k)}},$$

$$x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}.$$

Il metodo del gradiente consta essenzialmente di due fasi:

- 1 la scelta di una direzione di discesa $p^{(k)} = r^{(k)}$;
- 2 l'individuazione lungo tale direzione di un punto di minimo locale per ϕ ;

La seconda fase si può affrontare in modo indipendente dalla prima.

Quindi dato una generica direzione $p^{(k)}$ troviamo il valore α_k come quel valore di α che rende minimo

$$\phi(x^{(k)} + \alpha p^{(k)}).$$

Derivando la ϕ rispetto ad α ed imponendo la condizione di annullamento della derivata prima nel punto di minimo, si ottiene:

$$\alpha_k = \frac{p^{(k)T} r^{(k)}}{p^{(k)T} A p^{(k)}}.$$

Come si vede questa relazione si riduce a quella precedente se $p^{(k)} = r^{(k)}$, quindi come determiniamo la $p^{(k)}$ in modo da accelerare la convergenza?

Poichè

$$\begin{aligned} x^{(k+1)} &= x^{(k)} + \alpha_k p^{(k)} \\ Ax^{(k+1)} - b + b &= Ax^{(k)} - b + b + \alpha_k A p^{(k)} \\ r^{(k+1)} &= r^{(k)} - \alpha_k A p^{(k)} \end{aligned} \tag{1}$$

allora sostituendo la precedente espressione di α_k abbiamo

$$p^{(k)T} r^{(k+1)} = 0,$$

ovvero che il nuovo residuo risulta ortogonale alla direzione di ricerca.

Adesso moltiplicando (1) per $p^{(k)T}$ e sfruttando la relazione precedente, che ci dice che per passare alle iterazioni successive allora bisogna trovare una nuova direzione di ricerca $p^{(k+1)}$ in modo tale che

$$(Ap^{(j)})^T p^{(k+1)} = 0, \quad j = 0, 1, 2, \dots, k$$

Da un punto di vista pratico procediamo come segue.
Siano $p^{(0)}, p^{(1)}, \dots, p^{(k)}$ mutuamente coniugate ortogonali o A -ortogonali, ovvero:

$$(Ap^{(i)})^T p^{(j)} = 0, \quad i, j = 0, 1, 2, \dots, k, \quad i \neq j$$

con $k < n$. Come rendiamo la successione di direzioni di ricerca $p^{(0)}, p^{(1)}, \dots, p^{(k)}$ mutuamente A -ortogonali? A tal fine poniamo:

$$p^{(k+1)} = r^{(k+1)} - \beta_k p^{(k)}, \quad k = 0, 1, \dots$$

dove $p^{(0)} = r^{(0)}$ e le quantità $\beta_0, \beta_1, \dots$ sono da determinare.

I metodi di discesa

Metodo del gradiente coniugato CG

E' possibile trovare altre direzioni discendenti $p^{(k)}$ che assicura la convergenza del metodo in un numero inferiore di iterazioni? Quindi si va a definire una nuova discendenti p^{k+1} che sono A -ortogonali cioè

$$(Ap^{(j)})^T p^{(k+1)} = 0, \quad j = 0, 1, \dots, k.$$

Il metodo del gradiente coniugato è un metodo di discesa dove, posto $r^{(0)} = b - Ax^{(0)}$, $p^{(0)} = r^{(0)}$, k -esima iterazione è data da:

$$\alpha_k = \frac{(p^{(k)})^T r^{(k)}}{(p^{(k)})^T A p^{(k)}},$$

$$x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}$$

$$r^{(k+1)} = r^{(k)} - \alpha_k A p^{(k)},$$

$$\beta_k = \frac{(A p^{(k)})^T r^{(k+1)}}{(A p^{(k)})^T p^{(k)}}.$$

$$p^{(k+1)} = r^{(k+1)} - \beta_k p^{(k)}$$

Il metodo del gradiente coniugato: proprietà

Il metodo del gradiente coniugato ha molte proprietà particolari. Ne citiamo alcune.

Teorema: Se A è una matrice simmetrica e definita positiva di ordine n , allora il metodo del gradiente coniugato è convergente e fornisce in aritmetica esatta la soluzione del sistema $Ax = b$ in al massimo n iterazioni.

Questo teorema tradisce un po' le attese, in quanto in generale i calcoli non sono compiuti in aritmetica esatta, in molti casi della modellistica matematica n risulta essere molto alto.

Introduzione

- Il *gradiente coniugato* è un metodo efficace per problemi quadratici simmetrici.
- Il *metodo dei momenti* introduce una memoria nel gradiente per accelerare la convergenza.
- *ADAM* combina momenti e adaptive learning rates per un'ottimizzazione efficiente.

Sia $f : \mathbb{R}^n \rightarrow \mathbb{R}$, $n \geq 1$, il problema

$$\min_{x \in \Omega} f(x)$$

Un punto x^* è detto di *minimo globale (o locale)* se $f(x^*) < f(x)$ per ogni $x \in \Omega$.

problema di ottimizzazione, dove f è detta funzione di costo o funzione oggetto. Notiamo che valutare il massimo di una funzione f si conduce alla ricerca del minimo per $g = -f$.

Denotiamo come gradiente della funzione f , sufficientemente differenziabile in $\mathbb{R}$ nel punto $x \in \Omega$

$$\nabla f(x) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)$$

Introduzione-Metodi discendenti

Sia $f \in C^r(\mathbb{R})$, ed è limitata da sotto!

I metodi discendenti sono metodi iterativi in cui, per ogni $k \geq 0$, $x^{(k+1)}$, dipende da $x^{(k)}$, da un vettore $p^{(k)}$ dipendente da $\nabla f(x^{(k)})$ e da un adeguato parametro $\alpha_k \in \mathbb{R}$. Allora abbiamo, dato $x^{(0)}$, per $k = 1, 2, \dots$ il metodo funziona come segue:

- trovare una direzione: $p^{(k)} \in \mathbb{R}^n$,
- valutare lo step α_k ,
- settare $x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}$.
- il vettore $p^{(k)}$ deve essere una **direzione discendente** ovvero

$$(p^{(k)})^T \nabla f(x^{(k)}) < 0, \quad \text{se } \nabla f(x^{(k)}) \neq 0,$$

$$p^{(k)} = 0 \quad \text{se } \nabla f(x^{(k)}) = 0.$$

Una volta che $p^{(k)}$ è stato individuato si calcola il valore ottimale di $\alpha_k \in \mathbb{R}$, che garantisce la massima variazione di f lungo $p^{(k)}$. Di seguito alcuni esempi.

Analizziamo la relazione:

$$(p^{(k)})^T \nabla f(x^{(k)}) < 0, \quad \text{se } \nabla f(x^{(k)}) \neq 0,$$

$$D_p(f(x)) := \nabla f(x^{(k)})^T (p^{(k)})$$

Rappresenta la derivata direzionale di $f(x)$ lungo la direzione $p^{(k)}$.

Il gradiente di $f(x)$ punta sempre verso la direzione di massima crescita della funzione.

Se il prodotto scalare è negativo (< 0), significa che l'angolo tra la direzione scelta $p^{(k)}$ e il gradiente è ottuso (più di 90 gradi).

Muoversi in una direzione che "va contro" il gradiente garantisce che, almeno per passi molto piccoli, il valore di $f(x)$ scenda.

Quindi, imporre che il prodotto scalare sia minore di zero è la condizione matematica per definire $p^{(k)}$ come una direzione di discesa.

Direzioni di discesa

- ① direzione di Newton;
- ② direzioni quasi Newton;
- ③ direzioni del gradiente $p^{(k)} = -\nabla f(x^{(k)})$
- ④ direzioni del gradiente coniugato $p^{(0)} = -\nabla f(x^{(0)})$:

$$p^{(k+1)} = -\nabla f(x^{(k)}) - \beta_k p^{(k)}.$$

dove il parametro β_k può essere scelto a secondo dei criteri di discesa.

Metodo del Gradiente e del Gradiente Coniugato

- Direzioni del Gradiente $p^{(k)} = -\nabla f(x^{(k)})$
- Direzioni del gradiente coniugato:

$$p^{(0)} = -\nabla f(x^{(0)})$$

$$p^{(k+1)} = -\nabla f(x^{(k+1)}) - \beta_k p^{(k)}, \quad k \geq 0$$

Il parametro beta può essere scelto a secondo di diversi criteri, per esempio, **Fletcher-Reeves**

$$\beta_k^{FR} = -\frac{\|\nabla f(x^{(k)})\|^2}{\|\nabla f(x^{(k-1)})\|^2}$$

- Si applica per risolvere sistemi lineari simmetrici e definiti positivi.
- Migliora la discesa del gradiente scegliendo direzioni coniugate $p^{(k)}$.

Metodo dei Momenti

- Introduce una memoria nel gradiente per rendere più stabile l'ottimizzazione.
- L'aggiunta del termine di momentum mira ad accelerare la convergenza aggregando le informazioni dei gradienti precedenti. Ciò permette di ridurre la varianza degli aggiornamenti e di superare eventuali oscillazioni dovute alla natura stocastica dei campioni.
- Utilizza un parametro β per pesare il contributo degli step precedenti.

La formula per l'aggiornamento con momentum è:

$$\begin{aligned} m^{(k)} &= \beta m^{(k-1)} + (1 - \beta) \nabla f^{(k)}, \\ \theta^{(k+1)} &= \theta^{(k)} + \alpha m^{(k)}. \end{aligned}$$

- $m^{(k)}$, rappresenta il vettore di momentum all'iterata k ;
- β è il coefficiente di momentum (tipicamente compreso tra 0.9 e 0.99);
- α è il tasso di apprendimento.

Metodo ADAM (Adaptive Moment Estimation)

- Combina i momenti con una regolazione adattiva del learning rate.
- Utilizza due momenti: il primo momento (media) e il secondo momento (varianza).
- Update delle variabili:

$$\begin{aligned}m^{(k)} &= \beta_1 m^{(k-1)} + (1 - \beta_1) \nabla f^{(k)} \\v^{(k)} &= \beta_2 v^{(k-1)} + (1 - \beta_2) (\nabla f^{(k)})^2 \\x^{(k+1)} &= x^{(k)} - \frac{\alpha}{\sqrt{v^{(k)}} + \epsilon} m^{(k)}\end{aligned}$$

Esempio: Ottimizzazione con la funzione di Rosenbrock

La funzione di Rosenbrock è definita come:

$$f(x, y) = (a - x)^2 + b(y - x^2)^2,$$

dove di solito si imposta $a = 1$ e $b = 100$. Questa funzione ha un minimo globale in $(x, y) = (1, 1)$ ma presenta una valle stretta e curva che rende difficile la convergenza di molti metodi iterativi quando il punto di partenza è lontano dal minimo. Questa funzione viene utilizzata per valutare i metodi iterativi in particolare la loro convergenza dato che essa presenta una stretta curva di minimo.

Applicazione Nel Machine Learning: L'applicazione principale di questi algoritmi iterativi per il calcolo del minimo di una funzione differenziabile è l'ottimizzazione dei parametri di un modello di Machine Learning. Questo è particolarmente importante poiché i parametri del modello determinano la capacità di effettuare predizioni corrette.

Nel contesto del Machine Learning, l'obiettivo è minimizzare la **funzione di costo**, che rappresenta l'errore del modello. Ridurre tale funzione significa adattare il modello nel modo migliore possibile, aumentando la precisione delle predizioni.

Esempio: Ottimizzazione

- Neural network training richiede di risolvere problemi di ottimizzazione non vincolata della forma

$$\theta^* = \arg \min_{\theta} \mathcal{L}([X, Y], \hat{Y}; \theta), \quad \text{sogetto a} \quad \hat{Y} = f(X, \theta),$$

dove $[X, Y]$ sono le coppie di input-output del data set training, f è qualche Neural Network (NN) parametrizzata (con θ come parametri), θ^* sono i parametri ottimali, $\mathcal{L}$ è qualche *loss function*, e $\hat{Y}$ è l'output della NN per l'input X .

- L'obiettivo del training è trovare i parametri θ che minimizzano una *funzione di perdita* (o *loss function*), che misura la discrepanza tra l'output predetto $f(X, \theta)$ e l'output desiderato $\hat{Y}$. Esempi: The Mean Square Error (MSE) loss function è usata per la regressione, e la Categorical Cross-Entropy loss function per la classificazione di problemi.
- Il training consiste nell'ottimizzare i parametri θ per minimizzare la *loss function*. Questo viene fatto utilizzando metodi iterativi di ottimizzazione, come il gradiente discendente o sue varianti (ad esempio, SGD, Adam).

Passaggi dell'algoritmo ADAM.

Formulazione Matematica

Sia $\theta_t \in \mathbb{R}^n$ il vettore dei parametri al tempo t , e sia

$$g_t = \nabla_{\theta} \mathcal{L}([X, Y], \hat{Y}; \theta_t)$$

il gradiente della funzione di perdita rispetto ai parametri al tempo t .
L'algoritmo ADAM aggiorna i parametri iterativamente seguendo questi passaggi:

- 1 **Inizializzazione:** Inizializza i parametri θ_0 , inizializza le stime del primo momento m_t (media del gradiente) e del secondo momento v_t (varianza del gradiente):

$$m_0 = v_0 = 0,$$

m_t, v_t hanno le stesse dimensioni di θ_t .

Per ogni iterazione t :

- 2 **Aggiorna il gradiente**

$$g_t = \nabla_{\theta} \mathcal{L}([X, Y], \hat{Y}; \theta_t)$$

1 Aggiornamento del primo momento (momentum):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

dove $\beta_1 \in [0, 1)$ è un iperparametro che controlla il decadimento esponenziale del primo momento.

2 Aggiornamento del secondo momento (varianza):

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

dove $\beta_2 \in [0, 1)$ è un iperparametro che controlla il decadimento esponenziale del secondo momento.

3 Correzione del bias: Poiché m_t e v_t sono inizializzati a zero, possono essere soggetti a un bias iniziale, per correggerlo, si calcolano:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

4 Aggiornamento dei parametri:

$$\theta_{t+1} = \theta_t - h \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

dove h è il learning rate globale, ε è un piccolo valore (ad esempio, 10^{-8}) per evitare divisioni per zero.

Adam algorithm

ADAM può essere scritto come segue (ADAM with rescaling):

$$\hat{m}^k = \frac{m^k}{(1 - \beta_1^k)},$$

$$\hat{v}^k = \frac{v^k}{(1 - \beta_2^k)},$$

$$\theta^k = \theta^{k-1} - h \frac{\hat{m}^k}{\sqrt{\hat{v}^k + \varepsilon}},$$

con (ADAM semplice)

$$g^k = \nabla_{\theta} \mathcal{L}(t^k, \theta^{k-1}),$$

$$m^k = \beta_1 m^{k-1} + (1 - \beta_1) g^k,$$

$$v^k = \beta_2 v^{k-1} + (1 - \beta_2) (g^k)^2,$$

$$\theta^k = \theta^{k-1} - h \frac{m^k}{\sqrt{v^k + \varepsilon}},$$

e $\mathcal{L}(t^k, \theta^{k-1}) := \mathcal{L}([X, Y]_k, \hat{Y}_k; \theta^{k-1})$, dove $t^k = k h$.

In (Silva and Gazeau, 2020) l'algoritmo di ADAM ha la seguente forma.

Per ogni costante $\beta_1, \beta_2 \in (0, 1), \varepsilon > 0$ e valori iniziali θ_0 , $m_0 = v_0 = 0$ per ogni $k \geq 1$,

$$\begin{aligned}g^k &= \nabla_{\theta} \mathcal{L}(t^k, \theta^{k-1}), \\m^k &= \mu^k m^{k-1} + (1 - \mu^k) g^k, \\v^k &= \nu^k v^{k-1} + (1 - \nu^k) (g^k)^2, \\ \theta^k &= \theta^{k-1} - h \frac{m^k}{\sqrt{v^k} + \varepsilon}.\end{aligned}$$

con

$$\mu^k = \beta_1 \frac{1 - \beta_1^{k-1}}{(1 - \beta_1^k)}, \quad \nu^k = \beta_2 \frac{1 - \beta_2^{k-1}}{(1 - \beta_2^k)}.$$

ODEs ADAM

In (Silva and Gazeau, 2020) consideriamo la famiglia di tre parametri di un sistema differenziale ADAM:

$$\begin{aligned}\theta' &= -m/\sqrt{v} + \varepsilon, \\ m' &= \frac{g_1(t, \lambda, \alpha_1)}{(\nabla_{\theta} \mathcal{L}(t, \theta) - m)}, \\ v' &= \frac{g_2(t, \lambda, \alpha_2)}{([\nabla_{\theta} \mathcal{L}(t, \theta)]^2 - v)},\end{aligned}\tag{2}$$

con:

$$g_i(t, \lambda, \alpha_i) = \frac{1 - e^{-\lambda/\alpha_i}}{\lambda(1 - e^{-t\alpha_i})}, \quad i = 1, 2,$$

with $\lambda = h$ learning rate, $\beta_i = e^{-\lambda/\alpha_i}$, and $\alpha_i = -\lambda/\log \beta_i$ per $i = 1, 2$.

without scaling

For l'algoritmo di ADAM without rescaling abbiamo

$$\begin{aligned}\theta'(t) &= -\frac{m(t)}{\sqrt{v(t)} + \varepsilon}, \\ m'(t) &= d(t)\nabla_{\theta}\mathcal{L}(t, \theta(t)) - r(t)m(t), \\ v'(t) &= p(t)\nabla_{\theta}\mathcal{L}(t, \theta(t))^2 - q(t)v(t),\end{aligned}$$

dove

$$d(t) = r(t) = 1/\alpha_1, \quad p(t) = q(t) = 1/\alpha_2, \quad (3)$$

con $\beta_i = 1 - \lambda/\alpha_i$, for $i = 1, 2$.

The discrete-time model

Risolvere il problema di ottimizzazione in formulazione continua è, in effetti, come risolvere un sistema di equazioni differenziali ordinarie numericamente. Ci concentriamo in generale su ODEs della forma:

$$Y' = F(t, Y), \quad Y(0) = Y^0, \quad [0, T]$$

dove F rappresenta la dinamica non lineare, Y^0 le condizioni iniziali e T il valore del tempo finale.

Due metodi numerici ben noti per risolvere questa equazione sono lo schema di Eulero esplicito

$$Y^{k+1} = Y^k + hF(t^k, Y^k), \quad t^k = h k,$$

e lo schema di Eulero implicito

$$Y^{k+1} = Y^k + hF(t^{k+1}, Y^{k+1}), \quad t^k = h k,$$

Consideriamo la discretizzazione al primo ordine $k \geq 1$ di ADAM ODEs come

$$\begin{aligned}m^k &= m^{k-1} + hg_1(t^k, h, \alpha_1) (\nabla_{\theta} \mathcal{L}(t^k, \theta^{k-1}) - m^{k-1}), \\v^k &= v^{k-1} + hg_2(t^k, h, \alpha_2) (\nabla_{\theta} \mathcal{L}(t^k, \theta^{k-1})^2 - v^{k-1}), \\ \theta^k &= \theta^{k-1} - \frac{m^k}{\sqrt{v^k + \varepsilon}},\end{aligned}$$

dove $m^0 = v^0 = 0$ e $t^k = k h$ e con

$$\begin{aligned}g_1(t^k, h, \alpha_1) &= 1 - \beta_1 \frac{(1 - \beta_1^{k-1})}{1 - \beta_1^k} = 1 - \mu^k, \\g_2(t^k, h, \alpha_2) &= 1 - \beta_2 \frac{(1 - \beta_2^{k-1})}{1 - \beta_2^k} = 1 - \nu^k.\end{aligned}$$

Conclusioni

- ADAM è spesso usato in Machine Learning per neural network training, e si può provare che corrisponde ad un sistema di equazioni differenziali ordinarie nel limite di piccoli learning rates.
- L'idea è di determinare un metodo numerico di alto ordine per risolvere la ODE generando un'estensione dello scheme ADAM che performa meglio di quello classico su alcuni problemi di regressione e classificazione. Comunque metodi di alto ordine hanno più stadi da valutare quindi usano più valutazione di gradiente in ogni epoch (o time-step).
- Il nostro esperimento si focalizza sul determinare metodi time-stepping per minimizzare la funzione costo durante il training.